Chapter 14.2: UML Modeling

About this page:

Topics: How do I read / create UML class diagrams?

What Will I Do? Let's read and finally try to draw the chart in practice

Point Value: S250.

Indicative difficulty level: Moderate.

Rough Estimate of Workload:? 3-4 hours.

Related Projects: None.

It may be difficult to get an overall picture of a large number of classes and relationships between them. Similarly, it may be difficult to explain verbally how an algorithm in concurrent programming works or how the execution of a program proceeds from object to object when the methods call each other.

UML, Unified Modelling Language, is a general-purpose visualization and modelling language for software development, which provides one solution. The modelling language makes it easy to design larger systems at a coarse level, provide information quickly and in a small space, and even explain things to others than programmers. UML was created in the mid-90s to replace a number of different modelling languages developed to solve various visualization and modelling needs.

In practice, UML is a collection of different types of diagrams associated with software production processes. In this section, we will look at the *class diagram*, which is one of the most widely used UML presentations. A class diagram is a handy tool when designing a program, as it makes it easy to sketch out which classes should be created before the program code is written. It is also a quick way to describe and communicate the class structure to others - the reason why we use it in the project plan and in the document.

We illustrate in this section a part of possible features in class diagrams and only a small part of everything that UML contains. Interested people can easily find additional material on the net – the following includes a part of the definition of a class diagram that covers the most common

program design needs.

Project

NOTE. Project work information is only for Aalto students. MOOC students do not make the project.

Once the project topics have been published and selected, each student makes a preliminary plan for their program. The purpose is to create an _initial_ class structure for the work. In your technical design you will draw a UML class diagram. Once you've drawn the diagram, you can consider how some of your program's features work: How do class methods call each other when some activity is used? Does the diagram include everything that is necessary? Using UML in the project is not goal as such, but it makes it easier for an assistant to familiarize with the project, because the class diagram can give a quicker overview of the work than reading verbal descriptions covering ten classes. (It is still good to have the verbal description, though.)

Class diagram

Class

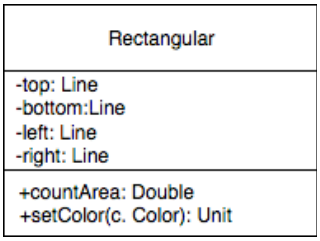
Below there is a simple Scala class Rectangular. It has, of course, four lines, which here are instances of the class Line. (To make a distinction between our example and class Rectangle in O1library we use name Rectangular for this class)

```
class Rectangular (top: Line, bottom: Line, left: Line, right: Line) {

  def countArea: Double = {
   // code ....
  }

  def setColor (c: Color): Unit = {
   // code ....
  }
}
```

In the class diagram, each class is represented by a rectangle with one to three segments. The class name is written in the top part of the diagram, and sometimes nothing else is needed. The middle segment (often omitted) can list the class instance variables. The lowest part includes class methods.



If you compare the diagram and the code with each other, you will notice that it is easy to read the same things as there are in the Scala code. The UML notation used to represent variables and methods is also very similar to the corresponding Scala code. The types of variables, parameters and return values are also listed in a familiar way after the colon.

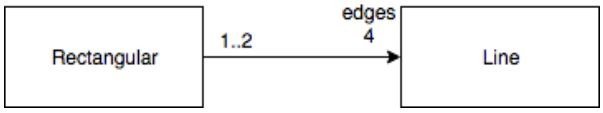
The minus and plus signs preceding the variables and the methods give information about the visibility. Minus corresponds to visibility declaration private in Scala, plus corresponds to visibility public. # matches the protected visibility and ~ package visibility. In Scala, visibility can, however, be adjusted more precisely than what can be clearly shown in the UML diagram.

The amount of details can be adjusted.

UML provides opportunities to express things either very precisely or on a quite general level. Only the name can be given for a class, if desired. Showing the visibility of variables and methods is not mandatory, if it is not useful for the use of the graph. We will soon look at the relationships between classes that are described with arrows. You can add more information about the quantities, relationship role, etc ... but this information can also be omitted if necessary.

Association

Relationships between classes are described in a class diagram with a line between two classes. Different arrowheads attached to the lines tell us what kind of relationship there is. The association relationship typically means that an instance variable of a class refers to an instance of another class. This relationship is described with a line that does not have any tips at all, or has an open tip like the one shown in the adjacent figure.



If you wish, you can list the **multiplicity** at either end of the line, that is, how many objects are on each side. In our example, the Rectangular has four lines, and if two Rectangulars happen to be adjacent, the same line may belong to two Rectangulars.

3	3 items
2..5	2-5 items
2..*	at least 2
*	any number

At the end of the line, we can also write the **role** of the line, that is, what is the significance of the object at this end of the relationship. In our example, the Lines are the *edges* of the Rectangular.

If you want to give a relationship a name, you could write a title in the middle of the line. We do not have an example of this in the material.

Last but perhaps the most important thing is the arrowhead. The **Navigability** arrowhead emphasizes the direction of the relationship between the classes. In our example, the Rectangular class refers to Line objects, but there are no (necessarily) references backwards from the Line objects. This is important when it comes to exploring what can be done with our designed model and how to get access to the content of another class from one class. If there are no arrowheads at all, it means that the direction of the relationship may be in either direction or in both directions.

You cannot submit this assignment

Exercise 1

This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 20 / 20

Show anyway

Soft landing. First, a few simple questions about what you just read.

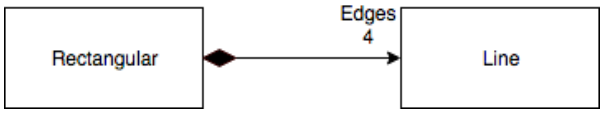
- [x] **UML class diagram does not have to include instance variables or methods.**
- [x] **If the association does not have any arrowheads, the relationship can be in either direction.**
- [x] **The roles can clarify what an association means in practice.**
- [ ] Multiplicity must always be included for the diagram.
- [ ] The class diagram always contains all the information needed to implement the class.

✔ Correct!

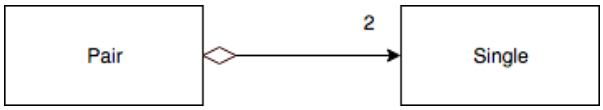
Submit

Composition and Aggregation

Composition and aggregation show that something consists of smaller parts. If some operations target to the ”main object”, they are in some way also targeted to its parts. Such a relationship is marked with a rhombus, which is marked at the ”heavier” end, that is, the instances of the class somehow contains other objects.



Composition: “The Rectangular consists of Lines: if the rectangular is removed, the lines will disappear.”



Aggregation: “The pair is made up of individuals: if a pair is discharged, the individuals will still be preserved.”

Aggregation is primarily intended to emphasize what kind of relationship it is about. Composition is a stricter version of aggregation, because it also indicates the existence of related entities. The above images with captions clarify the difference between these relationships. Stronger composites are labeled with a black rhombus, when weaker aggregation is labeled with an empty rhombus.

Composition and aggregation are additional information, which should be added when it adds value, but which can be replaced, if necessary, by a standard association. ”If in doubt - leave it out.”

Let us consider a small exercise.

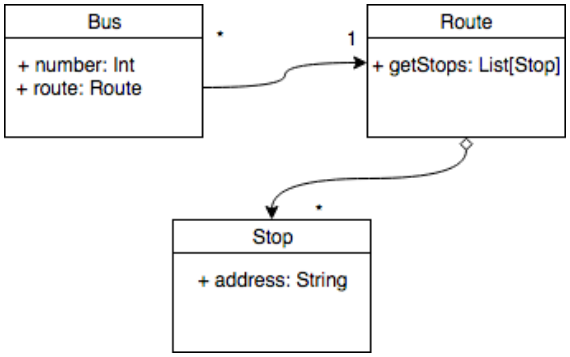
⚠

This course has been archived (Thursday, 1 June 2023, 12:00).
You cannot submit this assignment

Exercise 2 This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 20 / 20 Show anyway

The diagram below illustrates a bus and its route. Evaluate the correctness of the given statements in the light of the information in this section. Choose the correct claims:



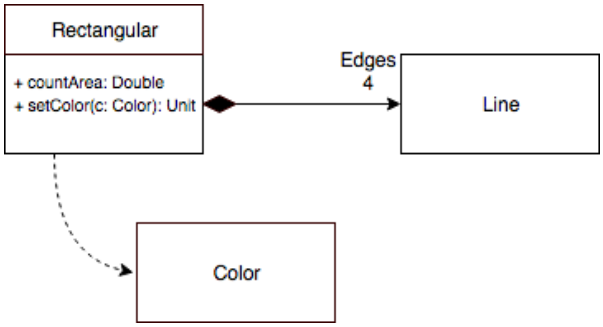
- [x] **The method in the Bus class cannot list which other Buses it is possible to change for along its route.**
- [ ] Removing the Route will lead to that Stops should be removed, too.
- [x] **It is possible that some Routes have no Buses at all.**
- [ ] Each Route can have only one Bus running on it.
- [x] **The method in the Bus class can list the addresses of the Stops on the Route it is running.**
- [x] **You may not be able to access from the Stop class to which Route it belongs.**
- [ ] Each Stop can only belong to one Route.

✔ Correct!

Submit

Dependency

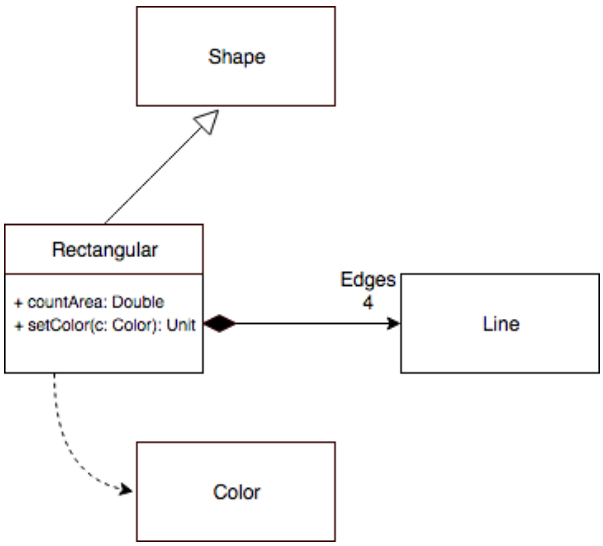
Dependency denotes a relationship when a class needs or uses another class as a part of its activity. Documenting such dependencies in the diagram will make it easier to modify the program in the future, because if any class changes, the diagram will show the classes that depend on it and their activity should be considered.



Dependence is presented like an association arrow where the arrow shaft is marked by a dashed line.

Inheritance

The inheritance relation is marked in the class diagram with an arrow with a non-filled triangle. The arrows always point to the super class.



From the above picture, it can be seen that the Rectangular is a Shape that, unlike the general Shape, has exactly four edges. The inheritance was discussed in section 7.3 (https://plus.cs.hut.fi/o1/2020/w07/ch03/) of the material and if you take a look at it, you realize that we also used UML notation there.

Analysis exercise

⚠ This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

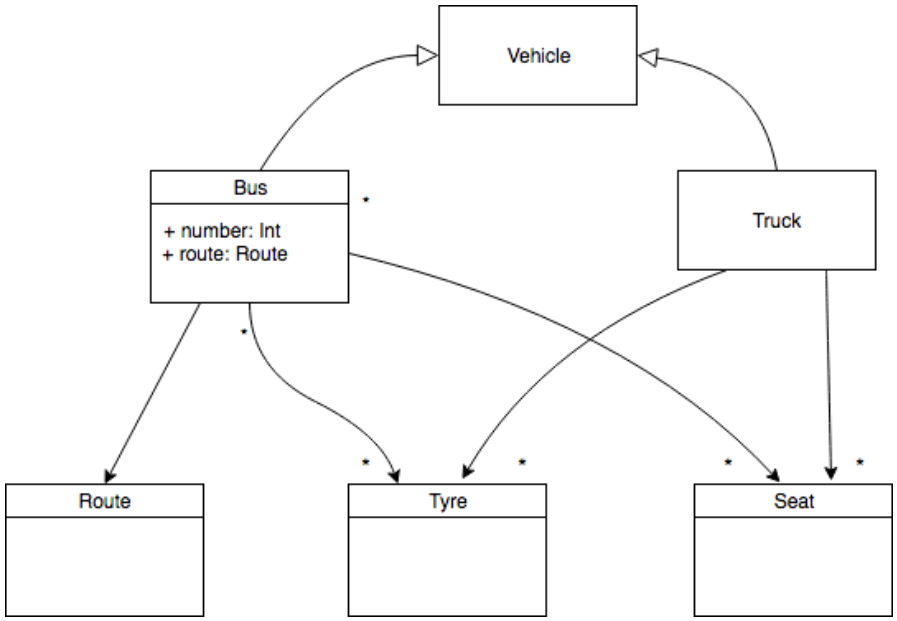
Exercise 3

This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 ~~20 / 20~~

Show anyway

The diagram below describes a bus and a truck. Assess the veracity of the given statements in the light of this section and the lessons you learned in the fall. Choose the correct claims:



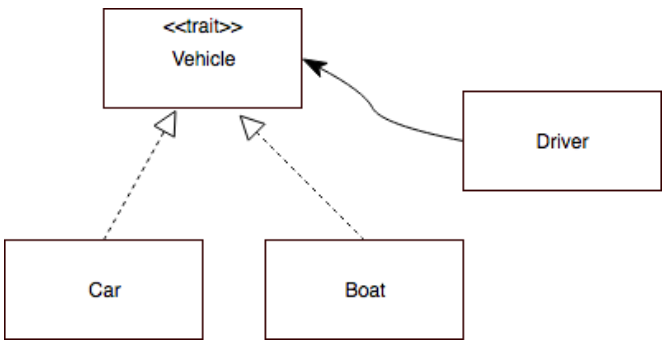
- [x] **The Bus does not necessarily have Seats.**
- [ ] All Vehicles will always have Tyres and Seats.
- [ ] All Vehicles have a Route
- [x] **Some Vehicles have a Route**
- [x] **Common features of Classes Bus and Truck (such as Tyres) should perhaps be moved to the super class.**
- [ ] A Vehicle object can simultaneously be both a Bus and a Truck.
- [ ] From the Wheel object you can see which Bus or Truck it belongs to but not which Vehicle it belongs to.

✔ Correct!

Submit

Trait and abstract class

UML does not, as such, include the concept of trait. However, it provides the possibility to write an attribute, called “stereotype”, which is labeled above the name of the class between double angles. In Scala, these attributes are << trait >> and << abstract >>. If there are also Java classes included in the diagram, there may also be << interface >>, which largely corresponds to a trait that does not have any implemented methods.

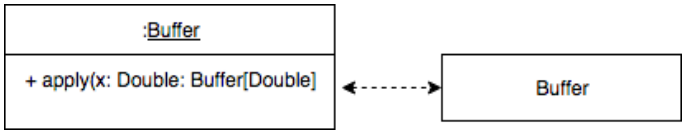


In the picture, the trait Vehicle describes everything possible to drive. Driver can handle all the things that have this feature. Categories Auto and Boat implement the abstract methods in the trait. Such a relationship between the abstract and the concrete class can be emphasized by using a dashed line.

If the diagram should present an abstract method, it is typically written in *italics*, which may be easier or more difficult to distinguish depending on the handwriting or font used.

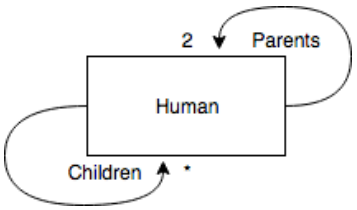
(Companion) objects

Individual objects can be presented in class diagrams using almost the same notation as the class. An object is distinguished from the class by having its title underlined and a possible colon in front of its name.

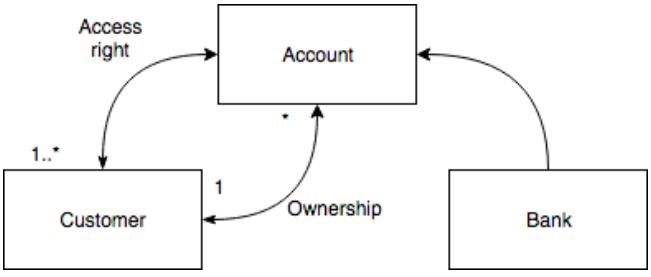


Structural recursion and more relationship between the entities

Structural recursion was discussed in Section 11.3 (https://plus.cs.hut.fi/o1/2020/w12/ch01/), which illustrated courses and their continuation courses, etc. The picture below shows a family tree diagram. An individual `Human` has relationships with other `Humans` who are either his or her children or parents. These relationships are separated by the roles ”children” and ”parents”.



There may also be more than one relationship between two objects. In this case, more relationships are drawn between the corresponding classes, and they can be distinguished from each other (as in our recursive example) by using either roles or naming the relationships.



In the diagram above, the bank customers have their own accounts, as well as other accounts where they have been granted access.

Another analysis exercise

Loading assignment...

Was that all?

Well, in practice, everything essential at least. In addition, there are still a number of smaller details that are needed less frequently, such as the restrictions between relationships or classes related to relationships, etc.

UML Modeling task

The best way to get to know UML is to do it yourself. Read the system description below and identify the essential classes you need for system modelling. You do not need to model the entire system in detail, but you can choose a **subset of the categories** for a more detailed look. We've given a lot more to ponder so that you can select some interesting modelling aspect.

At this stage of design, there is no need (read: you should not) to think about the user interface. Think about the features needed by the classes (methods, attribute variables) and especially the relationships between the classes. Draw the UML diagram of classes and return it in PDF format via the link below. Your diagram is evaluated manually and you get feedback, so do your work carefully. The assignment gets 170 points. The primary purpose of this assignment is to practice - the purpose is that the course project is not the first place where modelling can be trained.

The work can be done either with UML diagram software, some general-purpose drawing software, or even hand-made on paper where you can then scan the result into PDF. For example, draw.io (http://draw.io/) has been found to be a well-functioning online tool.

In practice, all new office programs produce PDF documents, so if you do not get your diagram in PDF format, attach it as an image to an (Open / Star / MS / Libre) Office document and save / output the result as PDF.

Verbal description: E-commerce

E-commerce site offers a wide range of products. The product information stores at least at the name, price, possible discounts (which may vary by customer and by date), product number, manufacturer, verbal description and possibly an image or multiple images. The customer can search for products either by product numbers or by searching the verbal descriptions of products or by browsing the products offered by the market for product segments. The product may belong to several product segments. For example, the micro-USB charging cable of a phone could belong to both the 'cables' product segment and the 'accessories' segment. Product segments may, in turn, include subcategories, for example, 'cameras' may include 'video cameras', 'SLR cameras', 'flash cameras', etc. Each manufacturer also constitutes their own product segment. This allows you to search for products such as "Samsung" and "product accessories" in the product segment. The store also offers multiple product packages, such as a specific camera, lens and stand. In addition to the products included in the product package, the total price of the package is also known.

There are different customers. The online service can be used as an unregistered or logged-in customer. Registered customers have a saved name, username and password and an email address, a possible phone number, and may have one or more delivery addresses and a credit card. Customers who are logged in will still be divided into private and business customers, for whom, for example, billing is subject to different tax rates, different discounts, etc. Business customers have additional business information, at least the company name.

The customer can add products to the basket and eventually make an order for these products. The order has the order number, its products, and information about the customer, the payment method (credit card, online payment, cash, etc.) and the delivery method (pick up, delivered to any R-kiosk, vending machine, delivery address, etc.). Finalizing subscriptions for logged-in customers can use pre-collected information about the customer. However, this information must also be changeable. Unregistered customers provide all the necessary information when finalizing their order. The subscription has a status that tells about the phase how the order is being processed, for example, that some products are still pending, the order has been compiled but not yet shipped, the order was shipped, the order was delivered. A logged-in customer can view the progress of their orders and old orders on the web service.

The store constantly maintains and develops its product range. Thus, it is fast and easy to add new products and product segments, remove old ones or change their data. You do not want to change the computer program that is running the online store when the product information changes.

The store can offer customers discounts on individual products or several related products. Discounts may vary based on customer type and customer's previous orders.

The store collects information about what products customers viewed and what they eventually bought. Based on this information, customers can be informed about each products, and what products other customers, who viewed this product, eventually bought.

Tips

Remember, this is the first time that you are practicing UML modelling. The purpose is not to produce a complete and comprehensive model. Thus, you should not try to describe everything above. Firstly, try to identify the key classes. You can use the noun method if you want, but remember that it is not advisable to use it mechanically. You can compile classes by theme and start from products, for example, and then proceed to customers, orders, etc. if you think it is necessary.

What kind of classes should be used? Can subclasses be utilized? Does your class structure allow implementing the functionality described above or is something difficult to accomplish with the class structure?

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

« Chapter 14.1: Planning (https://plus.cs.aalto.fi/studi…

Chapter 14.3: From Design to Implementation » (http…